



Programação Avançada 2025/26
05 Árvores | Algoritmos Recursivos

@Patricia Macedo & Bruno Silva

Algoritmos recursivos em Árvores

Na primeira aula sobre árvores demos como exemplo de algoritmos recursivos:

- travessia em pré-order
- tamanho -> `size`

Sempre que precisamos de implementar uma operação que tenha que percorrer a árvore usaremos como estratégia algoritmos recursivos.

Que outras operações nas árvores obrigam a percorrer os seus elementos ?

Algoritmo recursivo - árvore binária (Revisão)

Algoritmo pre-order numa árvore binária : [Raiz, subárvore esquerda, subárvore direita]

```
Algorithm: preOrder
    input: binaryTree
    output :None

BEGIN
    IF NOT(isEmpty(binaryTree) THEN
        WRITE(root(binaryTree))
        preOrder(leftTree(binaryTree))
        preOrder(rightTree(binaryTree))
    END IF
END
```

Algoritmo recursivo - árvore genérica(2)

Algoritmo pre-order : [Raiz, Sub-árvores]

- Usa-se um ciclo FOR para percorrer as sub-árvores da coleção de filhos

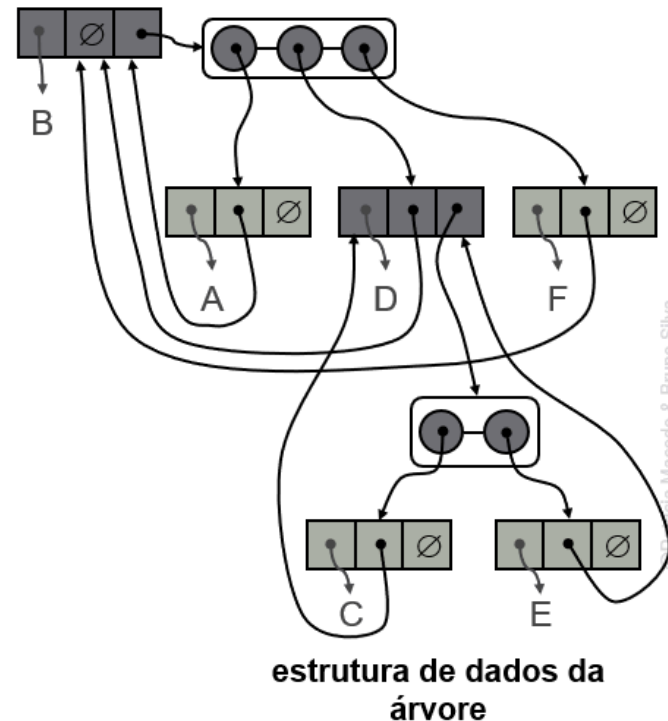
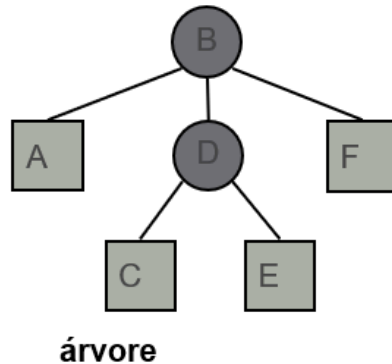
```
Algorithm: preOrder
    input: tree
    output :None

BEGIN
    IF NOT(isEmpty(tree) THEN
        WRITE(root(tree))
        FOR EACH child FROM children(tree)
            preOrder(child)
        END FOR
    END IF
END
```

Implementação em JAVA

Para implementarmos o algoritmo escrito em pseudocódigo teremos que fazer algumas adaptações em função da Estrutura de Dados selecionada.

- Um nó de uma árvore guarda
 - O elemento
 - Referência para o nó pai.
 - Uma lista de nós filhos.
- Um nó implementa o TAD Posicao.



Implementação em JAVA

Na ED abaixo um nó, guarda uma lista de **nós filhos** e não uma lista de sub-árvores.

```
public class TreeLinked<E> implements Tree<E> {  
  
    private TreeNode root;  
  
    private class TreeNode implements Position<E> {  
  
        private E element; // element stored at this node  
        private TreeNode parent; // adjacent node  
        private List<TreeNode> children; // children nodes  
  
        TreeNode(E element) {  
            this.element = element;  
            parent = null;  
            children = new ArrayList<>();  
        }  
    }  
}
```

Algoritmo recursivo - árvore genérica

Adaptação do algoritmo pre-order , considerando que o input é um nó e não uma árvore.

Algoritmo pre-order a partir de um nó-raiz : [Raiz, nós Filhos]

- Usa-se assim um ciclo FOR para percorrer os **nós filhos**

```
Algorithm: preOrder
    input: node
    output :None
BEGIN
    WRITE(node)
    FOR EACH child FROM children(node)
        process(child)
        preOrder(child)
    END FOR
END
```

Implementação em JAVA do algoritmo pre-order

```
public String toStringPreOrder() {  
    String str = "";  
    if (!isEmpty()) {  
        str = toStringPreOrder(root);  
    }  
    return str;  
}  
//recursive method  
private String toStringPreOrder(Position<E> position) {  
    String str = position.element().toString(); // visit (position)  
    for (Position<E> child : children(position)) {  
        str += "," + toStringPreOrder(child);  
    }  
    return str;  
}
```

- É **sempre** preciso termos um método auxiliar privado para implementar a recursividade. Este método tem como parametro Position, sobre o qual se avança na recursividade.
- Qual o critério de paragem no método recursivo?

Implementação em JAVA do algoritmo **size**

```
public String size() {  
    if (isEmpty()) return 0;  
    return size(root);  
}  
// método privado que implementa a recursividade  
private int size(Position<E> position) {  
    int count=1; // conta a raiz  
    for (Position<E> child : children(position)) {  
        count += size(child);  
    }  
    return count;  
}
```

Novamente o critério de paragem da recursividade está implícito:

- Lista de nós filhos vazia = nó externo

Quiz Time | 40 segundos

1. :Considere o método X da classe TreeLinked. O que retorna o método X?

-  O número de elementos da árvore
-  O grau da árvore
-  O número de nós internos da árvore
-  Nenhuma das anteriores

```
public int X() {  
    if (isEmpty()) return 0;  
    return X(root);  
}  
  
public int X(Position<E> pos) {  
    int c=0;  
    if(isExternal(pos)) return 1;  
    for (Position<E> w : children(pos))  
        c+=X(w);  
    return c;  
}
```

2.Considere o método `exists` que determina se um elemento existe na árvore. Qual a instrução que preenche corretamente o snippet1?

```
public boolean exists(E elem){  
    if (isEmpty()) return false;  
    //snippet1  
}  
private boolean exists(Position<E> pos, E elem){  
    boolean res=false;  
    for(Position<E> w: children(pos)){  
        //snippet2  
        return true;  
        res=exists(w,elem);  
    }  
    return res;  
}
```



`return exists(this,elem);`



`return exists(root,elem);`



`return true;`



Nenhuma das anteriores.

3.Considere o método `exists` que determina se um elemento existe na árvore. Qual a instrução que preenche corretamente o snippet2?

```
public boolean exists(E elem){  
    if (isEmpty()) return false;  
    //snippet1  
}  
private boolean exists(Position<E> pos, E elem){  
    boolean res=false;  
    for(Position<E> w: children(pos)){  
        //snippet2  
        return true;  
        res=exists(w,elem);  
    }  
    return res;  
}
```



`if(pos.element().equals(elem) )`



`if(w==elem)`



`if(w.element().equals(elem))`



Nenhuma das anteriores.

Exercício

Partindo do projeto iniciado na aula anterior (
https://github.com/estsetubal-pa/ADTTree_Template)

- Forneça o código dos métodos por implementar, i.e., os que estão a lançar `NotImplementedException` ;
- Compile e teste o programa fornecido.
- Implemente o método `public boolean exists(E elem)` que verifica se um elemento existe.

Treino em casa

Considere o algoritmo Breath-first para percorrer a árvore em largura, por níveis.

```
BFS(Tree)
```

```
  Coloque a raiz da árvore na fila
```

```
  Enquanto a fila não está vazia faça:
```

```
    seja n o primeiro nó da fila
```

```
    processe n
```

```
    para todo o f nó filho de n
```

```
      coloque f na fila
```

Implemente na classe `TreeLinked` o método Breath-first que devolve uma `Colecao` Iterável com os elementos da árvore ordenado segundo o algoritmo Breath-first.

Soluções Quiz Time 🧐

1. : O que retorna o método X?

👉 Nenhuma das anteriores

2. :Qual a instrução que preenche corretamente o snippet1?

✌️ `return exists(root,elem);`

3. :Qual a instrução que preenche corretamente o snippet2?:

👋 `if(w.element().equals(elem))`